UNIVERSITY OF THE WITWATERSRAND

Johannesburg, South Africa

Predictive Maintenance of the ATLAS Tile Calorimeter Using Machine Learning and NLP

Nicholas Perikli

University of the Witwatersrand

Technical Report based on the MSc dissertation submitted to the University of the Witwatersrand

August 2026

Abstract

The ATLAS Tile Calorimeter (TileCal) relies on a Detector Control System (DCS) to monitor and protect detector hardware. This study tests whether archived TileCal DCS alarms can also support predictive maintenance. Error-level alarm logs from selected Run-3 data-taking periods in 2022–2023 were cleaned using targeted natural-language-processing (NLP) and rule-based extraction, mapped to detector partitions, hardware classes, modules and five alarm categories, and reorganised into spatial and temporal datasets. Spatial analysis identified concentrated alarm activity and four high-activity module–alarm targets. Long short-term memory (LSTM) models were then used for module-level alarm-count forecasting and partition-level dominant-alarm classification. Forecasting mean absolute percentage error ranged from 12.82% to 22.87%, with EBA Module 6 producing the strongest relative result. The raw classifier achieved 88% accuracy but was dominated by NO CONNECTION; after chattering reduction, accuracy was 75% while macro-F1 improved from 0.27 to 0.57. The results demonstrate a proof of concept for converting historical detector-control alarms into predictive information, while limited sample sizes, validation design and the absence of live DCS testing constrain operational conclusions.

Keywords: ATLAS; Tile Calorimeter; predictive maintenance; detector control system; NLP; machine learning; LSTM.

1 Introduction

CERN's Large Hadron Collider (LHC) accelerates counter-rotating proton beams to near light speed and can deliver bunch crossings every 25 ns. Its major experiments include ATLAS, CMS, ALICE and LHCb [1, 2]. ATLAS and CMS discovered the Higgs boson in 2012, confirming the scalar sector required for electroweak symmetry breaking [3, 4, 5]. The Standard Model is organised by the gauge symmetry $SU(3)_C \times SU(2)_L \times U(1)_Y$; the Higgs field acquires a non-zero vacuum value, while its potential $V(\Phi) = -\mu^2\Phi^\dagger\Phi + \lambda(\Phi^\dagger\Phi)^2$ generates Higgs self-interactions. Of particular interest is the trilinear Higgs self-coupling, probed through rare Higgs-pair production [5, 6].

Higgs-pair production is roughly three orders of magnitude rarer than single-Higgs production, motivating the High-Luminosity LHC (HL-LHC), which will greatly increase the integrated collision dataset [7]. The resulting physics programme depends on sustained detector availability and stable operation. TileCal, the hadronic Tile Calorimeter of ATLAS, is divided into four read-out partitions (EBA, EBC, LBA and LBC), each with 64 module positions [8, 9]. TileCal DCS is the TileCal branch of the experiment-wide ATLAS DCS and monitors voltages, cooling, hardware states and alarms through global, sub-detector and local control layers [10]. Figure 1 summarises the detector, module and control-system hierarchy used in this work.

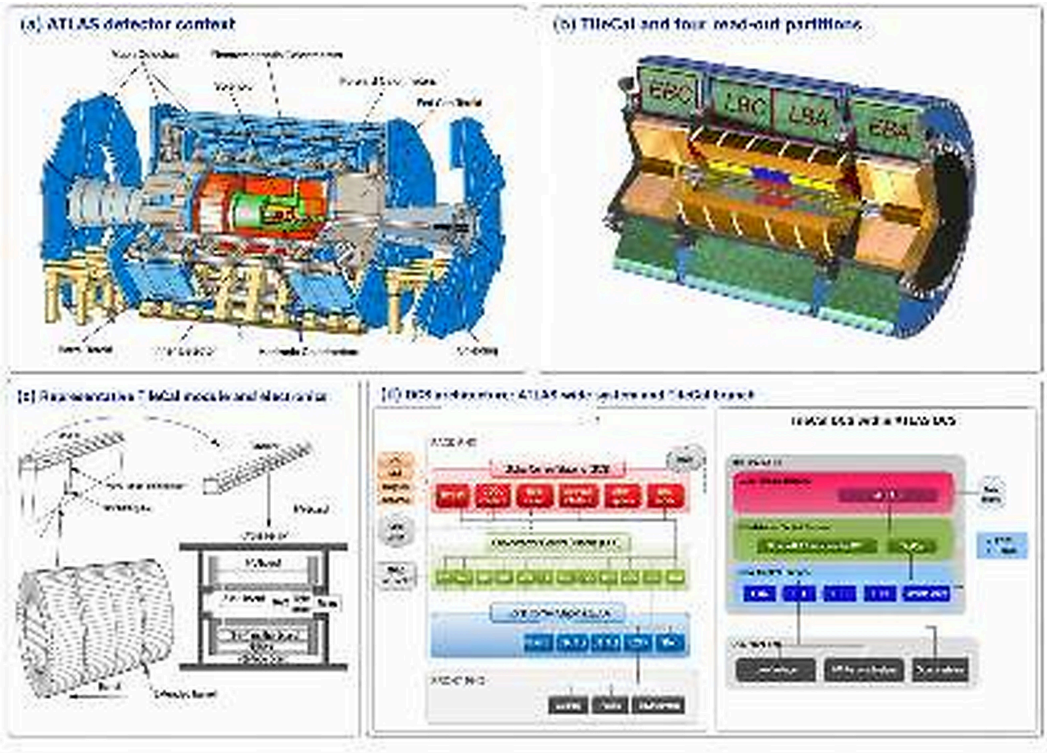

Figure 1: Detector and control-system context for the study: ATLAS, the four TileCal read-out partitions, representative module electronics, and the relationship between the ATLAS-wide DCS and TileCal DCS. Source images are adapted from ATLAS/TileCal/DCS documentation and the source MSc study [8, 9, 10, 15].

The existing DCS reacts when abnormal conditions or thresholds are detected; this study asks whether the historical alarm archive can provide an additional predictive layer. ML learns patterns from data for prediction or classification, while NLP extracts useful information from text. Earlier work by the author on COVID-19 vaccine hesitancy and Mpox stance detection developed the relevant experience in text preprocessing, annotation, model optimisation and evaluation [13, 14]. Here, those methods are transferred to semi-structured detector-control alarms. The study tests four linked propositions: alarm activity is temporally structured, spatially non-uniform, sufficiently informative for forecasting/classification, and potentially useful for predictive maintenance. The contribution is an end-to-end workflow that converts alarm text into detector-aware spatial

and temporal ML inputs, combines alarm-count forecasting with dominant-alarm classification, and links these outputs to the operational questions of where, how much and what type of alarm activity may occur next.

2 Background

2.1 Predictive maintenance, NLP and sequence modelling

Predictive maintenance uses operational history to anticipate developing faults before disruption, rather than responding only after failure [11]. TileCal alarms are suitable for this approach because they encode when an event occurred, where it originated and what condition was reported. Their text is semi-structured rather than free-form; targeted NLP therefore focuses on normalisation and rule-based extraction of partition, hardware class, module and alarm category rather than generic language modelling.

The temporal component is modelled with LSTMs [12]. An LSTM maintains a cell state and uses forget, input and output gates to regulate memory. For input x_t and previous hidden/cell states h_{t-1}, c_{t-1},

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \qquad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i),$$
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \qquad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \qquad h_t = o_t \odot \tanh(c_t).$$

Figure 2 illustrates this gated information flow. Forward LSTMs were used because the forecasting task requires causal use of past rather than future context.

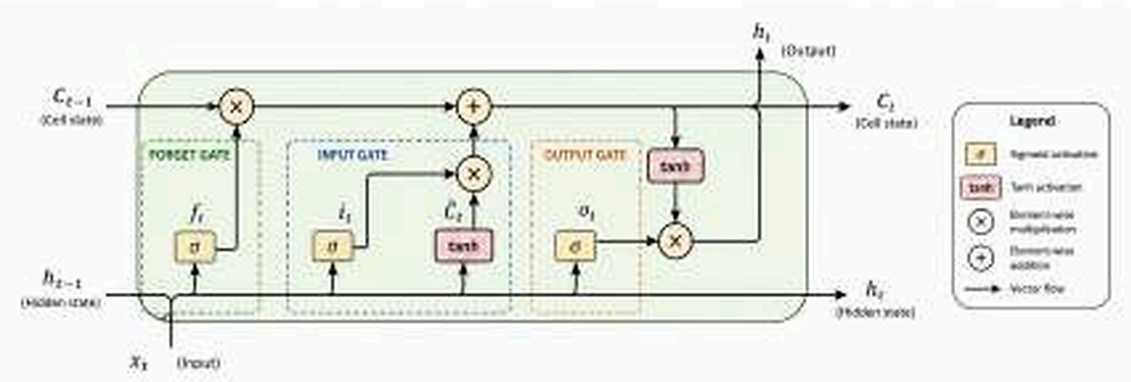

Figure 2: Gated information flow through an LSTM cell, showing the forget, input and output gates, candidate memory, cell state and hidden state. Redrawn for this report from the standard LSTM formulation [12].

3 Methodology

3.1 Data source and alarm preprocessing

TileCal DCS alarm logs were retained from active Run-3 data-taking periods in July–November 2022 and May–September 2023, excluding identified commissioning, testing and technical-stop intervals. The resulting dataset contained 18,476 alarm records [15]. Raw records comprised seven principal fields: Item, Abbreviation, Class, Alarm Text, Value, Timestamp and Directory. Only Error-level alarms were modelled. Text cleaning converted entries to lowercase, standardised equivalent terminology, removed redundant descriptors/special characters, and removed repeated words.

Structured variables were then extracted with regular expressions and TileCal naming rules. Partition identified EBA, EBC, LBA or LBC; Hardware Class identified components such as LVPS, Drawer, HVPS and Auxboard, and Module Number located the affected hardware. For `HVPS_N_channel_M`, where N is the HVPS unit (1–4) and M its channel (1–16), $M_{\text{module}} = M + (N-1)16$. For `Auxboard_N_channel_M`, where N is the Auxboard number (1–16) and M the channel (1–4), $M_{\text{module}} = N + (M-1)16$. The five standardised alarm classes were ALERT (E1), EMERGENCY MESSAGE RECEIVED/EMR (E2), NO CONNECTION (E3), NO TOGGLE (E4) and TRIPPED (E5).

Preprocessing procedure

1. Select approved data-taking periods and retain Error-level TileCal alarms.
2. Normalise the Item/alarm text.
3. Extract partition and hardware class.
4. Extract or reconstruct module number.
5. Consolidate alarm text into the five physical classes.
6. Order records chronologically and pass them to the time-series stage.

3.2 Time-series construction and model-ready data

Alarm events were grouped into retained chronological time bins; absent operational intervals were not artificially inserted as zero-count bins. For forecasting, data were separated by alarm type, partition and module, producing one count series per combination. Spatial hotspot analysis and temporal-behaviour plots were used to select one high-activity series per partition and its final temporal resolution. For classification, module subdivision was removed, alarms were grouped into 15-minute bins, counts were calculated for EBA/EBC/LBA/LBC, and the alarm category with the largest total count was assigned as the dominant class. No ties occurred.

Time-series construction procedure

1. Count alarms in each retained chronological interval.
2. Forecasting branch: create partition–module–alarm series, select high-activity candidates and choose series-specific time resolution.
3. Classification branch: calculate four partition counts per 15-minute bin, assign the dominant alarm type and order observations chronologically.

Model-ready preprocessing procedure

1. Preserve temporal order and split each dataset 70% training / 30% validation-evaluation.
2. Forecasting: fit Min-Max scaling on training data only, apply trailing smoothing where required, generate sliding-window sequences and inverse-transform predictions before final metrics.
3. Classification: retain a raw baseline, create a chattering-reduced dataset by consolidating runs of five consecutive observations with the same dominant class, fit standardisation on training data only, encode the five classes and generate sliding-window sequences.

Figure 3 condenses these preprocessing and modelling stages into the two final branches.

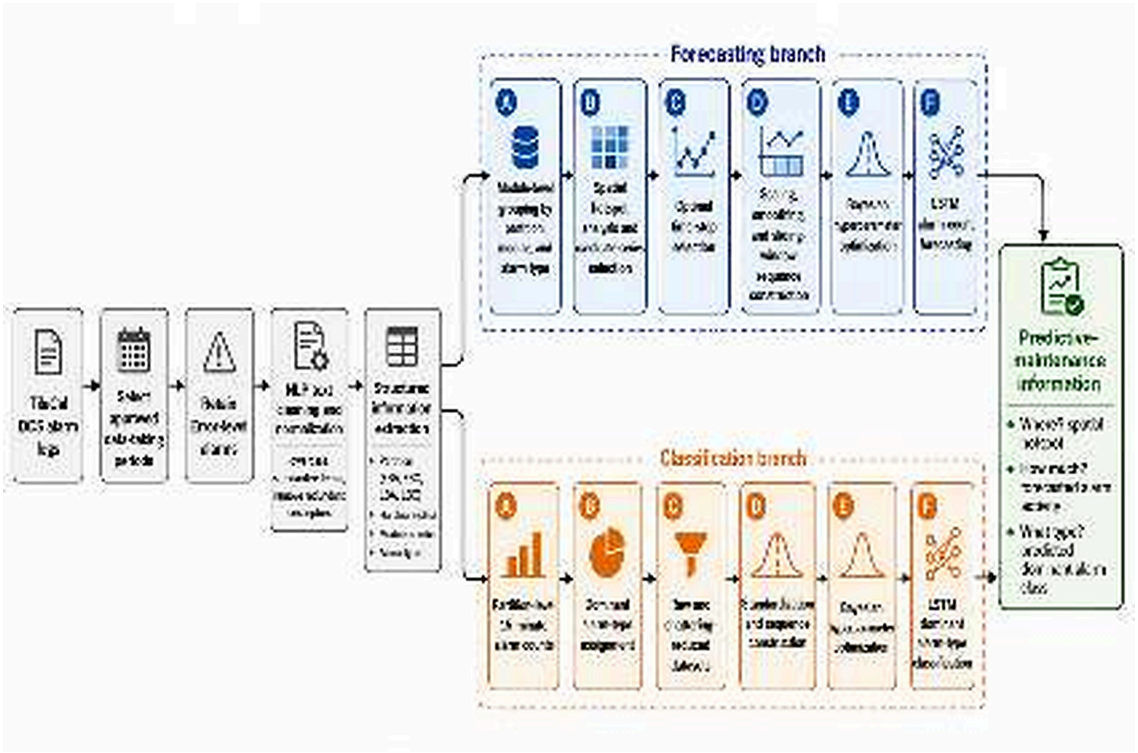

Figure 3: End-to-end TileCal alarm-analysis workflow, from DCS records and NLP extraction to module-level forecasting, partition-level classification and predictive-maintenance outputs.

3.3 Models, optimisation and evaluation

A separate PyTorch LSTM regression model was trained for each selected forecasting series [16]; the architecture used two recurrent layers with dropout and a dense regression output. Classification was implemented in TensorFlow with dense/dropout layers, two LSTMs and a five-class softmax output [17]. Raw and chattering-reduced classifiers were trained separately.

Hyperparameters were selected using Bayesian optimisation through Weights & Biases [18]. The search covered optimizer, batch size, LSTM units, learning rate, dropout, sequence length and stride, with weight decay used only for classification. Adam was retained for the final configurations [19]. Table 1 combines the search ranges and final settings in a single uniform table.

Parameter	Search space	EBA	EBC	LBA	LBC	Processed cls.	Raw cls.
Optimizer	Adam, Adamax, RMSprop, SGD	Adam	Adam	Adam	Adam	Adam	Adam
Batch size	8–512	76	121	97	112	136	256
LSTM units	8–512	59	35	98	45	32	128
Learning rate	10^{-7}–10^{-1}	.0082	.0003	.006	.026	.00024	.0001
Weight decay	10^{-8}–0.5 (classification only)	N/A	N/A	N/A	N/A	.2484	.01593
Dropout	0.01–0.75	.046	.122	.415	.403	.096	.25
Sequence length	tuned per dataset	2	2	100	2	10	25
Stride	tuned per dataset	1	1	10	1	1	3

Table 1: Bayesian hyperparameter search space and selected configurations for the four forecasting and two classification models. Values reproduce the source study with the corrected weight-decay range [15].

Training used early stopping (patience 10). Forecasting was monitored by validation loss and classification tuning by validation accuracy. Forecasting performance was reported using MSE, RMSE, MAE and MAPE after inverse transformation to the original alarm-count scale; classification used accuracy, precision, recall, F1, macro-F1 and weighted-F1. Macro-F1 is emphasised because of strong class imbalance.

4 Results

4.1 Spatial and temporal structure

Alarm distributions were strongly partition-dependent. LBA contributed 63.20% of total alarm activity and 96.90% of LBA alarms were NO CONNECTION; ALERT dominated EBA (41.01%), EBC (50.47%) and LBC (53.69%). Table 2 gives the complete partition-level distribution.

Alarm	TileCal	EBA	EBC	LBA	LBC
EMR	2.92%	7.57%	0.00%	0.74%	7.63%
ALERT	18.26%	41.01%	50.47%	1.33%	53.69%
TRIPPED	1.16%	0.99%	9.43%	0.34%	2.28%
NO TOGGLE	13.16%	28.94%	26.65%	0.68%	7.92%
NO CONNECTION	64.50%	21.49%	13.44%	96.90%	28.47%
Partition contribution	–	17.48%	4.52%	63.20%	14.95%

Table 2: Distribution of the five alarm categories across TileCal and its four partitions during the selected data-taking periods [15].

At module level, the selected hotspots were EBA M6–NO TOGGLE (60 alarms), EBC M10–ALERT (64), LBA M48–NO CONNECTION (11,146) and LBC M2–ALERT (180). Figure 4 shows the spatial contrast and the selected forecasting targets.

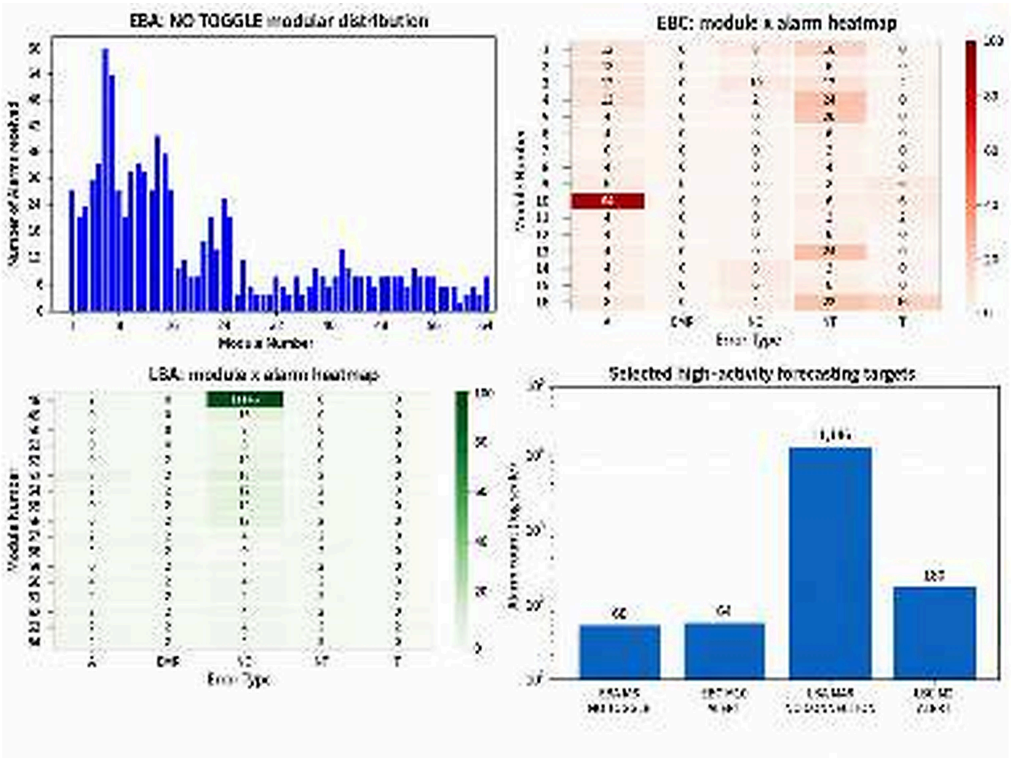

Figure 4: Spatial alarm activity and high-activity forecasting targets derived from the source analysis [15].

Temporal analysis further selected 15 min for EBA/LBC, 25 ms for EBC and 1 h for LBA, supporting the hypothesis that useful temporal structure occurs at series-specific scales. Figure 5 shows these patterns at their final modelling resolutions.

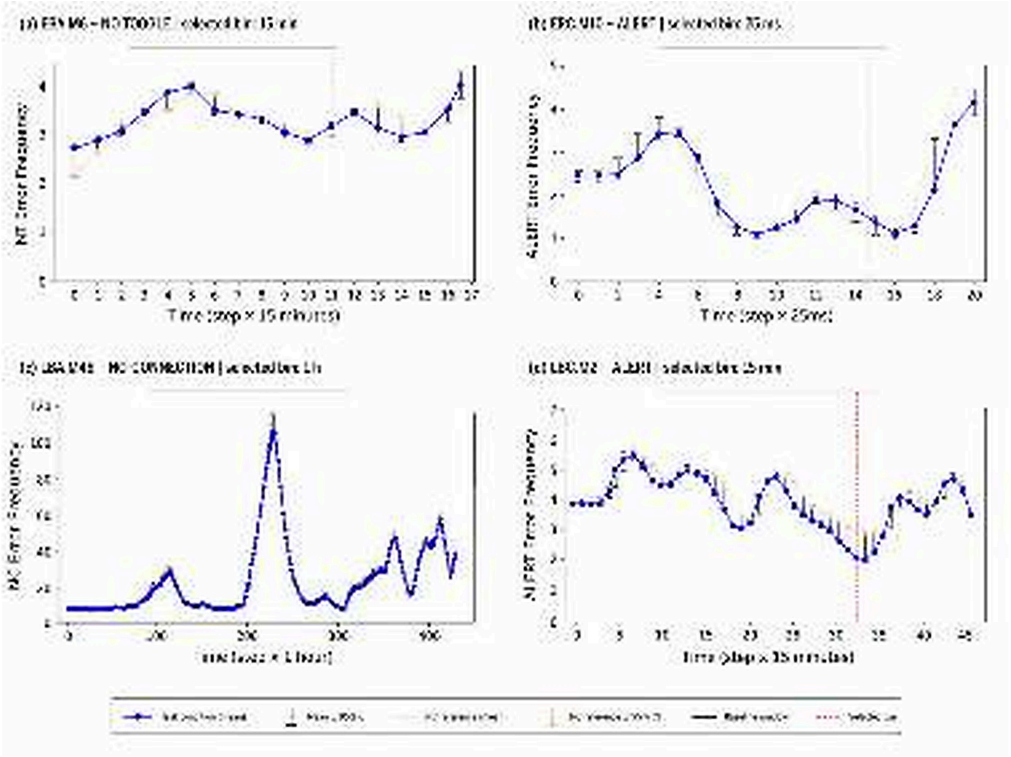

Figure 5: Temporal behaviour of the four selected module–alarm series at their final modelling resolutions: 15 min (EBA), 25 ms (EBC), 1 h (LBA) and 15 min (LBC).

4.2 Alarm-count forecasting

Forecasting MAPE ranged from 12.82% to 22.87%. EBA M6 ranked first (12.82%), followed by LBA M48 (16.83%), LBC M2 (21.92%) and EBC M10 (22.87%). Absolute errors must be interpreted within each series because count scales and temporal resolutions differ. Table 3 reports the complete results.

Target	Alarm	Time bin	MSE	RMSE	MAE	MAPE	Rank
EBA M6	NO TOGGLE	15 min	0.3195	0.5652	0.4413	12.82%	1
EBC M10	ALERT	25 ms	0.9044	0.9510	0.7701	22.87%	4
LBA M48	NO CONNECTION	1 h	26.4728	5.1452	4.2707	16.83%	2
LBC M2	ALERT	15 min	0.4940	0.7028	0.5947	21.92%	3

Table 3: Held-out alarm-count forecasting performance. Metrics were calculated after inverse transformation to the original alarm-count scale [15].

EBA provided the clearest forecasting result. LBC and EBC had low absolute errors but higher percentage errors, consistent with sparse/low-count intervals. LBA showed the second-lowest MAPE but requires additional caution because its sequence length of 100 and stride of 10 created strong overlap between neighbouring windows; the validation implication is discussed below.

4.3 Dominant alarm-type classification

The raw classifier achieved 88% accuracy but was heavily dominated by E3 NO CONNECTION (407/480 evaluation samples), giving macro-F1 = 0.27. After chattering reduction, accuracy fell to 75% while macro-F1 rose to 0.57, indicating more balanced performance. Table 4 gives the class-wise results. E1 F1 improved from 0.22 to 0.53, E2 from 0 to 0.40, E4 from 0 to 0.44 and E5 from 0.15 to 0.60; E3 decreased from 0.96 to 0.87. The rare-class gains remain uncertain because processed supports were only 7 (E2), 7 (E4) and 3 (E5).

	Raw				Processed			
Class	**P**	**R**	**F1**	**n**	**P**	**R**	**F1**	**n**
E1 ALERT	.58	.13	.22	50	.64	.45	.53	20
E2 EMR	.00	.00	.00	10	.67	.29	.40	7
E3 NO CONNECTION	.94	.98	.96	407	.80	.95	.87	59
E4 NO TOGGLE	.00	.00	.00	6	1.00	.29	.44	7
E5 TRIPPED	.09	.80	.15	7	.43	1.00	.60	3
Macro avg.	.32	.38	.27	480	.71	.59	.57	96
Weighted avg.	.87	.88	.86	480	.76	.75	.72	96
Accuracy	–	.88	–	480	–	.75	–	96

Table 4: Raw and chattering-reduced dominant alarm-type classification. Correct mapping: E1=ALERT, E2=EMR, E3=NO CONNECTION, E4=NO TOGGLE, E5=TRIPPED [15].

Figure 6 visualises the processed confusion matrix and the raw-versus-processed changes in precision, recall and F1.

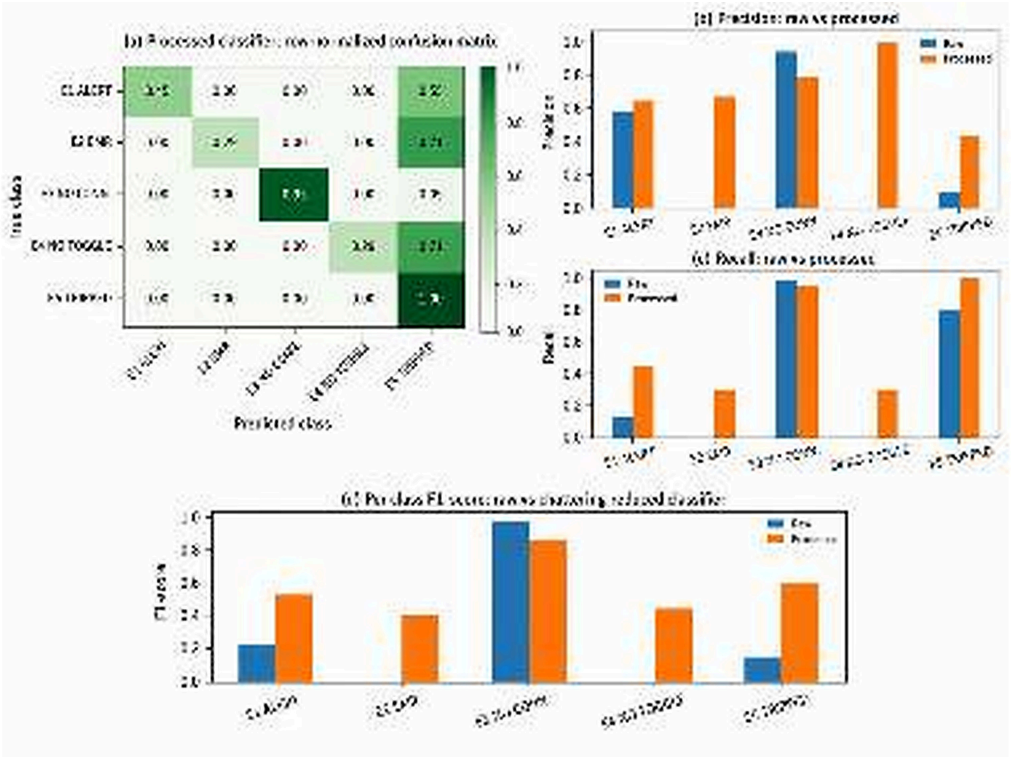

Figure 6: Effect of chattering reduction on dominant alarm-type classification: processed confusion matrix and raw-versus-processed class-level precision, recall and F1.

5 Discussion

The combined workflow provides three complementary outputs: spatial analysis identifies where alarm activity is concentrated, forecasting estimates how much activity may occur next, and classification estimates what type may dominate. This is more operationally useful than retrospective counts alone, but it is not yet an automated maintenance decision system. The strongest evidence is the clear spatial concentration, EBA forecasting performance, and the increase in classification macro-F1 after chattering reduction. The NLP-to-time-series transformation is itself a central contribution because the raw DCS archive was not collected as a predictive dataset.

Results also show that performance is dataset-dependent. MAPE is sensitive near zero, making low-count EBC/LBC series look worse in relative terms despite small absolute errors. Classification accuracy similarly overstates performance when one class dominates. The processed classifier is therefore more informative than the raw 88% accuracy figure, although its minority-class sample sizes are too small for strong generalisation claims.

6 Limitations and Future Work

The study used selected Error-level alarms only, consolidated raw descriptions into five classes, and evaluated models retrospectively. The chronological 70/30 split did not include a separate final test set. For LBA, sequence length 100 with stride 10 means neighbouring windows shared 90 of 100 observations; with no buffer around the split, training and evaluation windows near the boundary were not fully independent, so the reported LBA result may be optimistic. Chattering reduction also reduced the classification dataset from 1,926 to 368 observations. Detector configuration, calibration and run conditions may further affect transferability.

Future work should use explicit temporal buffers and a fully independent future test period, extend the dataset across additional operating periods, increase rare-class support, assess whether Warning/state variables provide useful precursor information, and evaluate the framework prospectively in a controlled near-real-time TileCal environment. No claim is made here that the current models reduce downtime or improve maintenance scheduling in live operation.

7 Conclusion

Historical TileCal DCS alarms contain exploitable spatial and temporal structure. The study converted semi-structured operational text into detector-aware ML datasets, identified strongly concentrated alarm hotspots, obtained forecasting MAPE of 12.82–22.87%, and improved classification macro-F1 from 0.27 to 0.57 after chattering reduction. These results support the technical feasibility of a predictive-maintenance layer that combines location, expected activity and likely alarm type. Stronger independent temporal validation and prospective DCS testing are required before operational deployment, but the framework provides a compact proof of concept relevant to reliable TileCal operation during the HL-LHC programme.

Data availability. The underlying TileCal DCS alarm records are not distributed with this report and remain subject to relevant CERN/ATLAS access policies.

Code availability. Cleaned analysis/model code may be released separately where collaboration and data-governance constraints permit; restricted DCS records will not be included.

Acknowledgements. The author acknowledges the University of the Witwatersrand, the Institute for Collider Particle Physics, and the wider CERN, ATLAS and TileCal communities. The interpretations in this report are the author's and do not imply official ATLAS or CERN endorsement.

References

[1] CERN, "Large Hadron Collider," CERN, accessed 27 Aug. 2026. `https://home.cern/science/accelerators/large-hadron-collider`.

[2] CERN, "LHC experiments," CERN, accessed 27 Aug. 2026. `https://home.cern/science/experiments`.

[3] ATLAS Collaboration, "Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC," *Phys. Lett. B*, vol. 716, pp. 1–29, 2012, doi:10.1016/j.physletb.2012.08.020.

[4] CMS Collaboration, "Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC," *Phys. Lett. B*, vol. 716, pp. 30–61, 2012, doi:10.1016/j.physletb.2012.08.021.

[5] S. Navas et al. (Particle Data Group), "Review of Particle Physics," *Phys. Rev. D*, vol. 110, 030001, 2024, doi:10.1103/PhysRevD.110.030001.

[6] ATLAS Collaboration, "Combination of Searches for Higgs Boson Pair Production in pp Collisions at $\sqrt{s}$ = 13 TeV with the ATLAS Detector," *Phys. Rev. Lett.*, vol. 133, 101801, 2024, doi:10.1103/PhysRevLett.133.101801.

[7] O. Aberle et al., *High-Luminosity Large Hadron Collider (HL-LHC): Technical Design Report*, CERN-2020-010, 2020, doi:10.23731/CYRM-2020-0010.

[8] ATLAS Collaboration, "The ATLAS Experiment at the CERN Large Hadron Collider," *JINST*, vol. 3, S08003, 2008, doi:10.1088/1748-0221/3/08/S08003.

[9] ATLAS Collaboration, "Readiness of the ATLAS Tile Calorimeter for LHC collisions," *Eur. Phys. J. C*, vol. 70, pp. 1193–1236, 2010, doi:10.1140/epjc/s10052-010-1508-y.

[10] A. Barriuso Poy et al., "The detector control system of the ATLAS experiment," *JINST*, vol. 3, P05006, 2008, doi:10.1088/1748-0221/3/05/P05006.

[11] T. P. Carvalho et al., "A systematic literature review of machine learning methods applied to predictive maintenance," *Comput. Ind. Eng.*, vol. 137, 106024, 2019, doi:10.1016/j.cie.2019.106024.

[12] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, pp. 1735–1780, 1997, doi:10.1162/neco.1997.9.8.1735.

[13] N. Perikli et al., "Detecting the Presence of COVID-19 Vaccination Hesitancy From South African Twitter Data Using Machine Learning," *IEEE Trans. Comput. Soc. Syst.*, 2025, doi:10.1109/TCSS.2025.3608636.

[14] N. Perikli et al., "Evaluating automatic annotation of lexicon-based models for stance detection of M-pox tweets from May 1st to Sep 5th, 2022," *PLOS Digit. Health*, vol. 3, e0000545, 2024, doi:10.1371/journal.pdig.0000545.

[15] N. Perikli, *Alert Signal Classification and Prediction Using Natural Language Processing for the Tile Calorimeter of ATLAS*, MSc dissertation, University of the Witwatersrand, 2025. `https://wiredspace.wits.ac.za/items/905fe8f1-3992-45be-a7c0-7acea4535b94`.

[16] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *NeurIPS*, vol. 32, pp. 8024–8035, 2019.

[17] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *OSDI 16*, pp. 265–283, 2016.

[18] L. Biewald, "Experiment Tracking with Weights and Biases," Weights & Biases, 2020. `https://www.wandb.com/`.

[19] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015, arXiv:1412.6980.